

PROJETO SO – FASE 2 (COM INTERNET / IA)

As respostas devem ser escritas apenas dentro dos parêntesis retos => [Resposta]

=====

IDENTIFICAÇÃO

=====

Grupo: [15044, 15040, 13869]

=====

1. REVISÃO DO MODELO INICIAL

=====

Foi possível melhorar a abordagem da Fase 1?

[] Sim [X] Não

Que melhorias foram introduzidas?

[

Melhoria 1: Otimização no numero de processos/threads. Inicialmente (na fase 1) foram definidas mais threads/processos do que eram necessarias.

]

=====

3. USO DE IA

=====

Indique as ferramentas usadas e os prompts inseridos:

[

Ferramenta 1 / Chat GPT 5.5:

Prompt 1: A primeira fase foi descrita assim(ficheiro de texto da fase 1). o projeto tem de seguir as regras do pdf e a estrutura inicial da screenshot. caso seja preciso melhorar a estrutura cria uma estrutura adequada do projeto. De seguida quero que crie um prompt para desenvolver o código do projeto. O prompt para o agente vai incluir todos os pdfs informativos de toda a matéria que nos demos de modo a não usar nada que não demos. Deves criar um prompt completo e de modo a abordar todos os pontos importantes do projeto seguindo a nossa estrutura inicial. O código deve ser comentado explicando as entradas, parâmetros, possíveis exceções e erros etc. o prompt deve incluir também uma tarefa para criar um plano de implementação, documentação do projeto e abordar estes pontos "Documento disponibilizado no canal preenchido (Projeto SO 25_26 Exploração Planetária — Modelo de Entrega Fase 2.txt). Este documento não necessita de ser preenchido sequencialmente. Assim, devem lê-lo integralmente antes de começar, para que possam completar as secções relevantes à medida que forem desenvolvendo o trabalho. Compilação de todos os materiais produzidos durante a aula. Devem incluir num ficheiro ZIP tudo o que produziram, como por exemplo: notas, diagramas, desenhos da solução, planos, código, possíveis soluções, entre outros. O material entregue hoje poderá sofrer alterações até à entrega final; ou seja, esta submissão corresponde apenas a uma abordagem inicial, que poderão rever / ajustar / alterar para a entrega final."

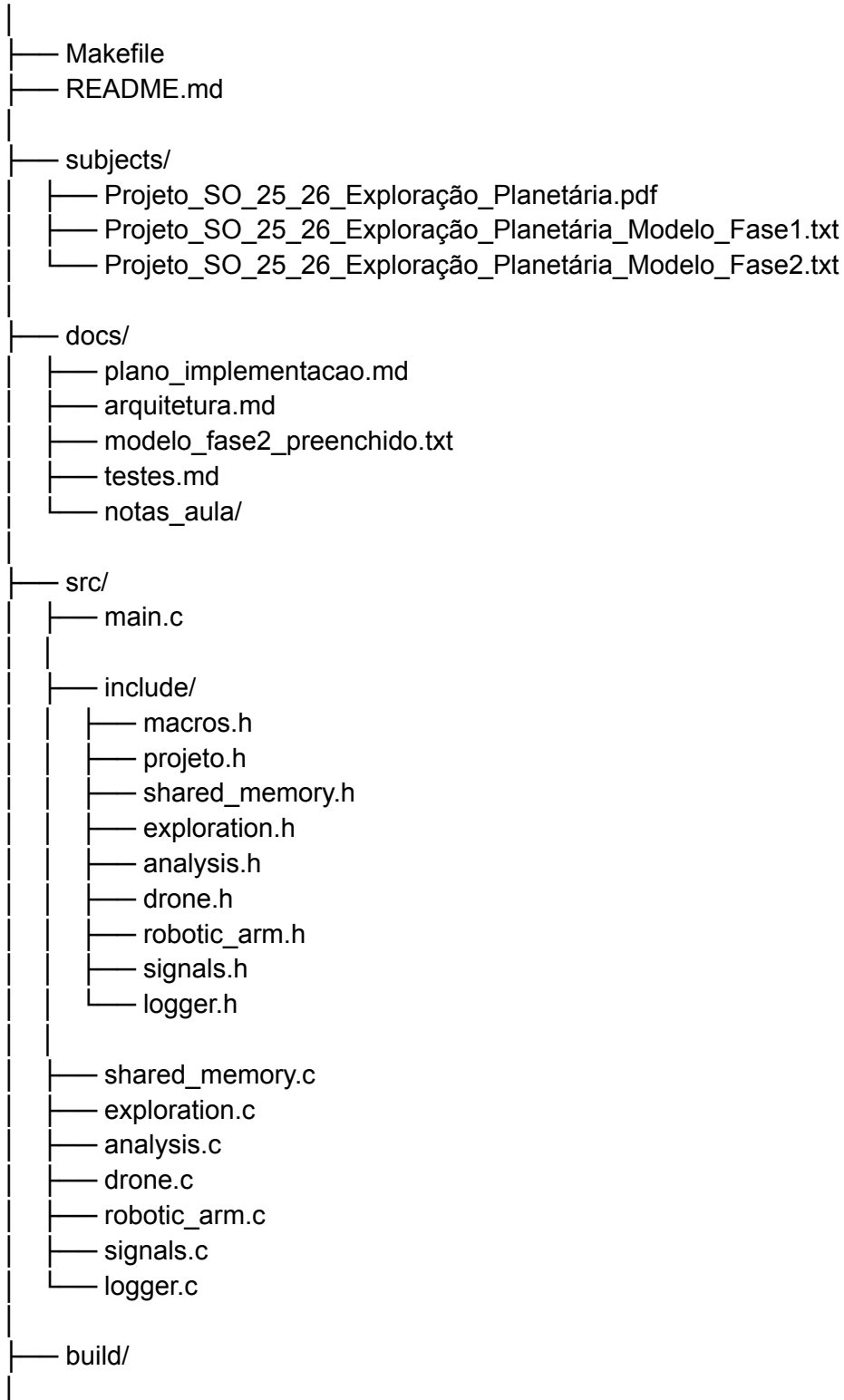
Ferramenta 2 / Claude Opus 4.6:

Prompt 2:

Estrutura recomendada do projeto

A tua estrutura atual pode ser melhorada para algo deste género:

SO_PROJETO/



```
|— bin/
|   |— estacao
|
|— entrega/
|   |— projeto_so_fase2.zip
```

Evita deixar no projeto final ficheiros como `main.o` ou executáveis soltos tipo `drone` na raiz. Isso deve ficar em `build/` e `bin/`, e ser apagado com `make clean`.

Prompt completo para dares ao agente

Copia isto e cola no agente que vai desenvolver o projeto:

Quero que desenvolvias o código completo de um projeto de Sistemas Operativos em C, para Linux/WSL Ubuntu, com base exclusivamente nos PDFs, apontamentos e materiais que vou anexar. Não uses internet, não uses bibliotecas externas e não uses conceitos que não estejam nos materiais fornecidos ou que não sejam POSIX/Linux básico dado em Sistemas Operativos.

O projeto chama-se “Estação Autónoma de Exploração Planetária” e simula uma estação robótica em Marte que recolhe e analisa amostras do solo.

Antes de escrever código, tens obrigatoriamente de:

1. Ler integralmente o enunciado do projeto.
2. Ler integralmente o documento “Projeto SO 25_26 Exploração Planetária — Modelo de Entrega Fase 2.txt”.
3. Ler os restantes PDFs/apontamentos da cadeira.
4. Ver a estrutura atual do projeto.
5. Criar primeiro um plano de implementação.
6. Só depois começar a implementar.

A solução deve seguir o enunciado e a modelação corrigida abaixo.

```
=====
1. OBJETIVO DO SISTEMA
=====
```

Desenvolver uma aplicação em C que simule uma estação autónoma de exploração planetária.

A estação tem:

- Uma frota de drones exploratórios.
- Um sistema de análise científica.
- Um tabuleiro de armazenamento temporário de amostras.

Os drones recolhem amostras e colocam-nas no tabuleiro.

O sistema de análise retira amostras do tabuleiro, analisa-as e depois descarta-as.

=====

2. ARQUITETURA OBRIGATÓRIA

=====

A aplicação deve ter os seguintes elementos:

1. Módulo Principal

- Implementação: processo principal.
- Responsabilidades:
 - Inicializar memória partilhada.
 - Inicializar mecanismos de sincronização.
 - Instalar handlers de sinais.
 - Criar o processo do Módulo de Exploração.
 - Criar o processo do Módulo de Análise Científica.
 - Tratar Ctrl-C/SIGINT para terminar o sistema.
 - Tratar Ctrl-Z/SIGTSTP para ativar/desativar a análise.
 - Esperar pelos processos filhos com wait/waitpid.
 - Libertar todos os recursos no fim.

2. Módulo de Exploração

- Implementação: processo filho.
- Responsabilidades:
 - Criar 3 threads de drones.
 - Coordenar a atividade dos drones.
 - Terminar corretamente quando o programa for encerrado.

3. Drones

- Implementação: threads dentro do processo de exploração.
- Número inicial: 3, definido com #define.
- Cada drone:
 - Recolhe uma amostra a cada 5 segundos, valor definido com #define.
 - Espera caso o tabuleiro esteja cheio.
 - Deposita uma amostra no tabuleiro quando houver espaço.
 - Deve imprimir mensagens informativas sobre as suas ações.

4. Módulo de Análise Científica

- Implementação: processo filho.
- Responsabilidades:
 - Criar 2 threads de braços/aparelhos de análise.
 - Gerir a análise científica.
 - Respeitar o estado ativo/desativo controlado por Ctrl-Z/SIGTSTP.
 - Terminar corretamente quando o programa for encerrado.

5. Braços robóticos / aparelhos de análise

- Implementação: threads dentro do processo de análise científica.
- Número: 2, definido com #define.

- Cada braço:
 - Retira apenas 1 amostra de cada vez do tabuleiro.
 - Espera se o tabuleiro estiver vazio.
 - Fica em standby se a análise estiver desativada.
 - Analisa a amostra durante um tempo aleatório entre 1 e 3 segundos.
 - Descarta a amostra após análise.
 - Deve imprimir mensagens informativas sobre as suas ações.

Não criar um processo separado para cada drone nem para cada braço, a menos que o enunciado ou os materiais da cadeira obriguem a isso. A estrutura pretendida é:

- Processo principal
- Processo de exploração
 - 3 threads de drones
- Processo de análise científica
 - 2 threads de braços robóticos

=====

3. CONSTANTES OBRIGATÓRIAS

=====

Criar as constantes num ficheiro de configuração, por exemplo src/include/macros.h.

Usar pelo menos:

```
#define BOARD_CAPACITY 10
#define NUM_DRONES 3
#define NUM_ANALYZERS 2
#define DRONE_DELIVERY_TIME 5
#define ANALYSIS_MIN_TIME 1
#define ANALYSIS_MAX_TIME 3
```

Também podem existir constantes para:

- Nome da shared memory.
- Tamanho máximo de mensagens.
- Códigos de erro.
- Estados booleanos.
- Nomes de módulos.

=====

4. MEMÓRIA PARTILHADA

=====

Deve ser usada memória partilhada para os dados comuns entre processos.

Dados que devem estar na memória partilhada:

- Tabuleiro de amostras.
- Número atual de amostras no tabuleiro.
- Próximo ID de amostra.

- Estado do sistema de análise: ativo/desativo.
- Estado de terminação do programa.
- Contadores estatísticos, se fizer sentido:
 - Total de amostras produzidas.
 - Total de amostras depositadas.
 - Total de amostras analisadas.
 - Total de esperas por tabuleiro cheio.
 - Total de esperas por tabuleiro vazio.
- Mecanismos de sincronização partilhados, caso sejam usados `pthread_mutex_t` e `pthread_cond_t` com `PTHREAD_PROCESS_SHARED`.

A estrutura pode ser semelhante a:

```
typedef struct {
    int id;
    int drone_id;
    time_t created_at;
} sample_t;

typedef struct {
    sample_t board[BOARD_CAPACITY];
    int count;
    int in;
    int out;

    int next_sample_id;
    int analysis_active;
    int terminate;

    int total_created;
    int total_deposited;
    int total_analyzed;

    pthread_mutex_t mutex;
    pthread_cond_t can_deposit;
    pthread_cond_t can_analyze;
} shared_data_t;
```

A estrutura exata pode ser adaptada, mas deve garantir:

- Capacidade máxima de 10 amostras.
- Inserção e remoção sincronizadas.
- Ausência de condições de corrida.
- Consistência no número de amostras.

```
=====
5. SINCRONIZAÇÃO
=====
```

A solução não pode ter esperas ativas.

Usar mecanismos POSIX adequados, preferencialmente:

- pthread_mutex_t com atributo PTHREAD_PROCESS_SHARED.
- pthread_cond_t com atributo PTHREAD_PROCESS_SHARED.

Ou, se os materiais da cadeira indicarem preferencialmente semáforos:

- semáforos POSIX.
- mutex para secção crítica.
- semáforo de espaços livres.
- semáforo de amostras disponíveis.

A solução tem de garantir:

1. Drones não depositam se o tabuleiro estiver cheio.
2. Braços não retiram se o tabuleiro estiver vazio.
3. Braços não analisam se o sistema de análise estiver desativado.
4. Não há duas threads a alterar o tabuleiro simultaneamente.
5. Não há inconsistência no contador de amostras.
6. Não há deadlock.
7. Não há starvation evidente.
8. Não há espera ativa com ciclos while a consumir CPU.

Caso uses condition variables, a lógica deve ser deste tipo:

Drones:

- Lock mutex.
- Enquanto tabuleiro cheio e sistema não estiver a terminar:
 - pthread_cond_wait(can_deposit)
- Se estiver a terminar:
 - unlock e sair.
- Depositar amostra.
- Atualizar contadores.
- pthread_cond_signal ou pthread_cond_broadcast(can_analyze).
- Unlock mutex.

Braços:

- Lock mutex.
- Enquanto sistema não estiver a terminar e:
 - tabuleiro vazio OU análise desativada:
 - pthread_cond_wait(can_analyze)
- Se estiver a terminar:
 - unlock e sair.
- Retirar amostra.
- Atualizar contadores.
- pthread_cond_signal ou pthread_cond_broadcast(can_deposit).
- Unlock mutex.
- Analisar fora da secção crítica.
- Descartar amostra.

A análise deve ocorrer fora da secção crítica para não bloquear o tabuleiro enquanto uma amostra está a ser analisada.

=====

6. SINAIS

=====

Implementar tratamento de sinais:

1. Ctrl-C / SIGINT

- Deve terminar o programa de forma limpa.
- O processo principal deve:
 - Assinalar na memória partilhada que o sistema está a terminar.
 - Acordar threads bloqueadas em condition variables ou semáforos.
 - Esperar pelos processos filhos.
 - Libertar memória partilhada.
 - Destruir mutexes/condition variables/semaphores.
 - Fazer shm_unlink, se for usada POSIX shared memory.
 - Evitar processos zombie.

2. Ctrl-Z / SIGTSTP

- Não deve suspender o programa.
- Deve alternar o estado do sistema de análise:
 - Se estiver ativo, passa a desativo.
 - Se estiver desativo, passa a ativo.
- Quando desativo:
 - Os drones continuam a recolher/depositar amostras.
 - Os braços robóticos ficam em standby.
 - Se o tabuleiro encher, os drones ficam bloqueados à espera de espaço.
- Quando reativado:
 - Os braços voltam a retirar e analisar amostras disponíveis.

A implementação dos handlers deve respeitar boas práticas:

- Não fazer lógica complexa diretamente dentro do handler.
- Usar flags do tipo volatile sig_atomic_t, se necessário.
- Fazer o processamento real no fluxo principal quando possível.
- Garantir que o programa não fica bloqueado ao terminar.

=====

7. OUTPUT / LOGS

=====

Todas as ações relevantes devem produzir mensagens claras no ecrã.

Usar tabulações conforme o enunciado:

- Módulo Principal: "\t"
- Drones: "\t\t\t"

- Módulos de análise: "\t\t\t\t\t"

Exemplos de mensagens:

Módulo principal:

"\t[MAIN] Sistema iniciado."

"\t[MAIN] Shared memory criada."

"\t[MAIN] SIGTSTP recebido. Sistema de análise DESATIVADO."

"\t[MAIN] SIGINT recebido. A terminar sistema..."

"\t[MAIN] Recursos libertados. Fim."

Drones:

"\t\t\t[DRONE 1] Amostra 7 recolhida."

"\t\t\t[DRONE 1] Amostra 7 depositada no tabuleiro. Ocupação: 4/10."

"\t\t\t[DRONE 2] Tabuleiro cheio. A aguardar espaço..."

Análise:

"\t\t\t\t\t[ANALISADOR 1] Amostra 7 retirada do tabuleiro. Ocupação: 3/10."

"\t\t\t\t\t[ANALISADOR 1] A analisar amostra 7 durante 2 segundos."

"\t\t\t\t\t[ANALISADOR 1] Amostra 7 analisada e descartada."

"\t\t\t\t\t[ANALISADOR 2] Sistema de análise em standby."

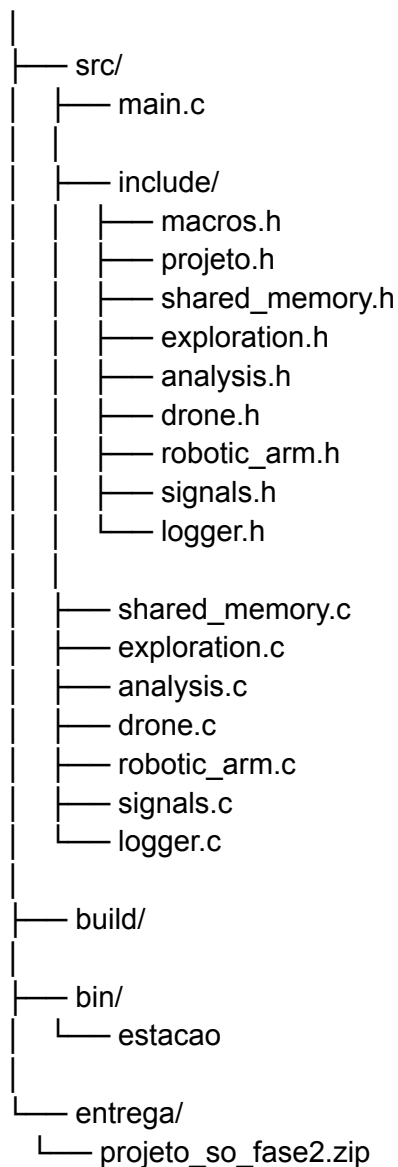
Evitar output confuso. Se necessário, proteger prints com mutex próprio ou usar uma função logger centralizada.

=====
8. ESTRUTURA DO PROJETO
=====

Se a estrutura atual estiver incompleta ou mal organizada, reorganizar para uma estrutura limpa:

SO_PROJETO/

```
|
|— Makefile
|— README.md
|
|— subjects/
|   |— Projeto_SO_25_26_Exploração_Planetária.pdf
|   |— Projeto_SO_25_26_Exploração_Planetária_Modelo_Fase1.txt
|   |— Projeto_SO_25_26_Exploração_Planetária_Modelo_Fase2.txt
|
|— docs/
|   |— plano_implementacao.md
|   |— arquitetura.md
|   |— modelo_fase2_preenchido.txt
|   |— testes.md
|   |— notas_aula/
```



Não deixar ficheiros .o nem executáveis soltos na raiz.

Os objetos devem ir para build/.

O executável deve ir para bin/.

9. MAKEFILE

Criar um Makefile robusto com pelo menos:

- make
- make all
- make run
- make clean
- make zip

A compilação deve usar flags rigorosas:

CC = gcc
CFLAGS = -Wall -Wextra -Werror -pedantic -std=c11 -D_XOPEN_SOURCE=700 -pthread
LDLIBS = -pthread

Se a implementação de shared memory/semaphores no ambiente exigir -lrt, adiciona apenas se necessário.

O projeto deve compilar sem:

- Erros.
- Warnings.

O comando principal deve ser algo como:

make
make run

E o programa deve executar:

./bin/estacao

=====

10. COMENTÁRIOS NO CÓDIGO

=====

O código deve estar bem comentado, mas sem exagero.

Cada função relevante deve ter comentário a explicar:

- Objetivo da função.
- Parâmetros de entrada.
- Valor de retorno.
- Possíveis erros.
- Recursos que cria/liberta, se aplicável.

Exemplo:

```
/**  
 * Inicializa a memória partilhada usada pelos processos.  
 *  
 * Cria ou abre a região de memória partilhada, define o seu tamanho,  
 * faz mmap e inicializa os dados principais do sistema.  
 *  
 * @return ponteiro para shared_data_t em caso de sucesso;  
 *      NULL em caso de erro.  
 *  
 * Possíveis erros:  
 * - shm_open falhar.  
 * - ftruncate falhar.
```

* - mmap falhar.
*/

Também comentar partes críticas:

- Secções críticas.
- Espera por condition variables/semaphores.
- Tratamento de sinais.
- Libertação de recursos.

Não escrever comentários inúteis como:

i++; // incrementa i

=====

11. TRATAMENTO DE ERROS

=====

Todas as chamadas importantes devem ser verificadas.

Verificar erros em:

- fork
- waitpid
- pthread_create
- pthread_join
- pthread_mutex_init
- pthread_cond_init
- pthread_mutex_lock
- pthread_mutex_unlock
- pthread_cond_wait
- pthread_cond_signal/broadcast
- shm_open
- ftruncate
- mmap
- munmap
- shm_unlink
- sigaction
- sem_init/sem_wait/sem_post/sem_destroy, se forem usados semáforos
- malloc/free, se forem usados

Em caso de erro:

- Imprimir mensagem clara com perror ou fprintf(stderr, ...).
- Libertar recursos já criados sempre que possível.
- Evitar deixar memória partilhada ou semáforos pendurados.
- Evitar processos zombie.

=====

12. DOCUMENTAÇÃO A CRIAR

=====

Além do código, criar documentação em docs/.

1. docs/plano_implementacao.md

Deve conter:

- Objetivo do projeto.
- Interpretação do enunciado.
- Entidades identificadas.
- Decisão processo/thread para cada entidade.
- Estrutura da memória partilhada.
- Mecanismos de sincronização escolhidos.
- Estratégia para SIGINT.
- Estratégia para SIGTSTP.
- Riscos previstos.
- Plano de tarefas por ordem de implementação.
- Possíveis melhorias futuras.

2. docs/arquitetura.md

Deve conter:

- Diagrama textual ou ASCII da arquitetura.
- Explicação dos processos.
- Explicação das threads.
- Explicação do fluxo de amostras:
 - Drone recolhe.
 - Drone espera espaço.
 - Drone deposita.
 - Braço espera amostra e sistema ativo.
 - Braço retira.
 - Braço analisa.
 - Braço descarta.

3. docs/testes.md

Deve conter testes manuais:

- Compilação.
- Execução normal.
- Verificar se drones depositam a cada 5 segundos.
- Verificar se análise demora entre 1 e 3 segundos.
- Pressionar Ctrl-Z e confirmar que a análise entra em standby.
- Pressionar Ctrl-Z novamente e confirmar que a análise retoma.
- Deixar a análise desativada até o tabuleiro encher.
- Confirmar que drones esperam quando o tabuleiro está cheio.
- Pressionar Ctrl-C e confirmar terminação limpa.
- Confirmar que não ficam processos zombie.
- Confirmar que não há warnings na compilação.

4. docs/modelo_fase2_preenchido.txt

Preencher o modelo disponibilizado pelo professor.

Ler o documento todo antes de preencher.

Não é obrigatório preencher sequencialmente.

Completar as secções relevantes à medida que o projeto for sendo desenvolvido.

Manter o formato original do documento.

Se o modelo usar respostas entre parêntesis retos [], escrever apenas dentro desses campos.

5. README.md

Deve conter:

- Nome do projeto.
- Descrição curta.
- Requisitos de compilação.
- Como compilar.
- Como executar.
- Como limpar.
- Como criar ZIP.
- Como usar Ctrl-Z e Ctrl-C.
- Explicação resumida da arquitetura.
- Lista de ficheiros principais.

=====

13. MATERIAL PARA ZIP

=====

Criar uma pasta entrega/ com um ZIP contendo tudo o que foi produzido.

O ZIP deve incluir:

- Código fonte.
- Makefile.
- README.md.
- docs/plano_implementacao.md.
- docs/arquitetura.md.
- docs/testes.md.
- docs/modelo_fase2_preenchido.txt.
- Notas produzidas durante a aula.
- Diagramas/desenhos da solução, se existirem.
- Possíveis soluções/rascunhos relevantes.
- O modelo da fase 1 preenchido, se existir.
- O modelo da fase 2 preenchido.

Não incluir:

- Ficheiros .o.
- Executáveis temporários fora de bin/.
- Ficheiros desnecessários do sistema.
- Pastas de cache.

- Ficheiros gerados sem utilidade.

=====

14. PLANO DE IMPLEMENTAÇÃO OBRIGATÓRIO

=====

Antes de programar, escrever um plano com esta ordem:

Fase 1 — Análise

- Ler enunciado.
- Ler modelo de entrega fase 2.
- Identificar entidades.
- Confirmar processos e threads.
- Confirmar recursos partilhados.
- Escolher mecanismos de sincronização.

Fase 2 — Estrutura

- Reorganizar diretórios.
- Criar headers.
- Criar Makefile.
- Criar README inicial.

Fase 3 — Memória partilhada

- Implementar shared_memory.c/h.
- Criar estrutura shared_data_t.
- Inicializar tabuleiro.
- Inicializar mutexes/condition variables ou semáforos.

Fase 4 — Processo principal

- Implementar main.c.
- Instalar sinais.
- Criar processos filhos.
- Esperar pelos processos.
- Libertar recursos.

Fase 5 — Módulo de exploração

- Implementar processo de exploração.
- Criar 3 threads de drones.
- Implementar ciclo de vida dos drones.

Fase 6 — Módulo de análise

- Implementar processo de análise científica.
- Criar 2 threads de braços.
- Implementar standby quando análise estiver desativada.
- Implementar análise com tempo aleatório entre 1 e 3 segundos.

Fase 7 — Sinais

- Implementar Ctrl-Z/SIGTSTP para ativar/desativar análise.

- Implementar Ctrl-C/SIGINT para terminar.
- Garantir que threads bloqueadas acordam ao terminar.

Fase 8 — Testes

- Testar execução normal.
- Testar tabuleiro cheio.
- Testar tabuleiro vazio.
- Testar Ctrl-Z.
- Testar Ctrl-C.
- Testar ausência de zombies.
- Testar make clean.
- Testar make zip.

Fase 9 — Documentação

- Preencher docs.
- Preencher modelo fase 2.
- Atualizar README.
- Criar ZIP final.

=====

15. REGRAS IMPORTANTES

=====

- Não usar espera ativa.
- Não usar sleep como substituto de sincronização.
- sleep só deve simular tempo de recolha/análise.
- Não usar bibliotecas externas.
- Não usar internet.
- Não usar código copiado de fontes externas.
- Não usar conceitos que não estejam nos materiais fornecidos.
- Não inventar requisitos que o enunciado não pede.
- Se houver uma decisão ambígua, documentar a decisão tomada.
- O código tem de compilar em WSL Ubuntu.
- O programa tem de compilar sem warnings.
- O programa tem de terminar limpo.
- O programa não pode deixar processos zombie.
- O programa não pode deixar shared memory/semaphores por libertar.
- O output tem de ser claro e legível.

=====

16. RESULTADO ESPERADO

=====

No fim, quero que entregues:

1. Código C completo.
2. Headers organizados.
3. Makefile funcional.

4. README.md.
5. docs/plano_implementacao.md.
6. docs/arquitetura.md.
7. docs/testes.md.
8. docs/modelo_fase2_preenchido.txt.
9. entrega/projeto_so_fase2.zip.
10. Explicação curta de como compilar e executar.

Antes de mostrar o código final, apresenta:

- A estrutura de ficheiros criada.
- O plano de implementação.
- As decisões técnicas principais.
- A correspondência entre requisitos do enunciado e implementação.

]

Das ferramentas de IA utilizadas, qual (ou quais) considera ter contribuído de forma mais útil para o desenvolvimento do trabalho?

[

Ferramenta 1: Chat GPT 5.5

Prompt 1:

Justificação 1: É útil para fazer tarefas mais técnicas como auxílio em escrever relatórios ou criação de prompts para agentes.

Ferramenta 2: Antigravity Claude Opus 4.6

Prompt 2:

Justificação 2: Dos melhores IDEs com integração de agentes de IA gratuitos com quota limitada.

]

Que limitações ou problemas foram identificados nas respostas IA obtidas?

[A solução da IA é demasiado confusa, há muita coisa repetida. O principal problema é criar a solução mais rápida possível, ou seja, não tem em conta a segurança da aplicação nem a otimização. O código por vezes (em projetos maiores como este) acaba por se tornar demasiado grande para o que é preciso.]

Houve sugestões dadas por IA que decidiram NÃO utilizar?

[

Sugestão: Sugeriu não usar semáforos e apenas mutex.

Porque foi rejeitada: Não usar semáforos e apenas mutex faz com que o projeto tenha usado espera ativa que era estritamente proibido no enunciado do projeto.

Problema identificado: Espera ativa.

]

Como validaram se as sugestões da IA estavam corretas?

[

Estratégia 1: Verificar o run do projeto.

Estratégia 2: Partilhar o código com outras IAs incluindo o enunciado, tentando identificar novos problemas.

Estratégia 3: Identificar leaks, processos zombies, ou outros problemas em tempo de execução e pós execução.

]

O que aprenderam após utilizar IA que não tinham considerado inicialmente?

[

Aprendizagem 1: Temos de especificar tudo o que queremos ao detalhe, ou seja, apesar da IA fazer temos de ter conhecimento sobre a arquitetura de projetos, funcionamento de processos/threads e outros conceitos sobre sistemas operativos, pois se não explicarmos o problema corretamente e a arquitetura corretamente.

Aprendizagem 2: Apesar de extremamente rápido, as suas soluções podem ser “as mais rápidas” e não resolve o problema da melhor forma.

]

Que alternativas consideraram e que acabaram por não ser escolhidas? (e porquê?)

[

Alternativa: Implementar uma UI live.

Vantagem: Podemos ver o que está a acontecer de forma mais visual em vez de um terminal, que pode parecer mais verboso.

Desvantagem: Utilizar uma UI live em C pode ser complexo.

Motivo da rejeição:

]

Que limitações ou fragilidades ainda existem na solução proposta?

[Escalabilidade]

Classifique a confiança que teve nas respostas da IA:

[] Baixa

[X] Média

[] Elevada

Porquê?

[Apesar de ser muito útil e otimizar de forma significativa a nossa produtividade, não podemos confiar cegamente no resultado final gerado pela IA, levando ainda a uma revisão completa do código. Só depois de uma análise crítica sobre todo o código gerado e possíveis alterações é ...]

=====

4. DISCUSSÃO

=====

Indique o grau de confiança na solução proposta:

[] Baixo

☒ Médio
☐ Elevado

Justifique:

[A solução fornecida pelo IA não foi fidedigna ao que era pedido no enunciado, sem pensamento crítico teríamos um trabalho que não cumpria os objetivos mínimos do projeto.]